

Convolutions causales partagées et objets lexicaux résiduels pour la modélisation du langage sans attention

Anonymous authors

Paper under double-blind review

Abstract

Nous étudions si un petit modèle de langage causal peut séparer le transport de l'information séquentielle du calcul propre aux tokens, sans attention fondée sur des requêtes, clés et valeurs. Le modèle proposé emploie une pile partagée de convolutions causales depthwise dilatées pour le mélange séquentiel, un objet de faible rang indexé par l'identité du token et partagé entre les couches, ainsi que d'étroits résidus de micro-experts déterministes. L'état de décodage obtenu conserve une taille fixe par rapport à la longueur du contexte généré et ne nécessite ni matrice d'attention ni cache clés-valeurs croissant. Nous définissons des contrôles appariés qui isolent le mélangeur séquentiel, l'objet lexical et le résidu de micro-experts à environ 100 millions de paramètres. Ce brouillon omet volontairement les résultats numériques principaux jusqu'à l'achèvement des entraînements préenregistrés sur un milliard de tokens et des évaluations tenues à part. L'étude est conçue pour rapporter à la fois la qualité du modèle de langage à nombre de tokens apparié, le débit d'entraînement propre au matériel et le débit de décodage incrémental.

1 Introduction

Les modèles de langage fondés sur l'attention couplent deux fonctions : transporter l'information entre les positions et sélectionner dynamiquement les positions qui interagissent. Ce couplage est expressif, mais le déploiement autorégressif conserve généralement un cache clés-valeurs (KV) dont la taille croît avec la longueur du contexte. Nous posons une question empirique plus étroite : à l'échelle de 100 millions de paramètres, un substrat causal régulier peut-il transporter le contexte tandis que de petits résidus conditionnés par les tokens assurent la spécialisation ?

Notre conception suit un principe de chemin partagé complété par des résidus. Chaque token traverse le même chemin de calcul dense et causal. L'identité du token ne module que des branches résiduelles étroites. Le mélangeur séquentiel partagé est une pile de convolutions causales depthwise dont les dilatations suivent un cycle ; il ne contient ni projection de requêtes, clés ou valeurs, ni softmax d'attention, ni opération d'attention positionnelle. Une table lexicale de faible rang partagée et des micro-experts déterministes ajustent ensuite la transformation commune.

Les contributions visées sont empiriques et ne prétendent pas que chaque composant pris séparément est nouveau :

- une intégration contrôlée d'un substrat convolutionnel causal partagé, d'objets lexicaux résiduels liés et de micro-experts déterministes étroits ;
- des ablations à nombre de paramètres apparié qui séparent les effets du mélange séquentiel et de la spécialisation lexicale ;
- une formulation exacte du décodage incrémental à état de taille fixe, testée par équivalence avec l'évaluation de la séquence complète ;
- une présentation transparente de la qualité, du débit d'entraînement, du débit de décodage, de la mémoire et des ablations ayant échoué à l'échelle de 100 millions de paramètres.

Nous ne prétendons pas remplacer universellement l'attention. L'étude porte sur une échelle, un tokenizer, un corpus et un calendrier de champs réceptifs précis.

2 Méthode

2.1 Mélangeur séquentiel causal partagé

Soit $\mathbf{H} \in \mathbb{R}^{B \times T \times d}$ le tenseur des états cachés normalisés. La couche ℓ utilise une convolution causale depthwise de largeur de noyau k et de dilatation

$$\delta_\ell = 2^{\ell \bmod C}, \quad (1)$$

où C désigne la longueur du cycle de dilatation. Après un remplissage à gauche de $(k-1)\delta_\ell$, le mélangeur s'écrit

$$\mathbf{C}_\ell = \text{DWConv}_{k, \delta_\ell}(\mathbf{H}), \quad (2)$$

$$\mathbf{M}_\ell = \mathbf{W}_o [\text{SiLU}(\mathbf{W}_g \mathbf{C}_\ell) \odot (\mathbf{W}_u \mathbf{C}_\ell)]. \quad (3)$$

Tous les tokens utilisent les mêmes paramètres et le même calcul. Aucune projection QKV ni matrice d'attention n'est instanciée.

Pour le décodage incrémental, la couche ℓ ne conserve que ses $(k-1)\delta_\ell$ derniers vecteurs d'entrée. L'état total par séquence est

$$\mathbf{S} = d \sum_{\ell=0}^{L-1} (k-1)\delta_\ell, \quad (4)$$

indépendamment du nombre de tokens déjà générés. En mode inférence, l'implémentation met à jour sur place des buffers d'état préalloués.

2.2 Objet lexical résiduel partagé

La transformation feed-forward partagée est un SwiGLU dense,

$$\mathbf{F}(\mathbf{h}) = \mathbf{W}_d [\text{SiLU}(\mathbf{W}_g \mathbf{h}) \odot \mathbf{W}_u \mathbf{h}]. \quad (5)$$

Pour l'identité de token x_t , une table partagée $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times r}$ fournit une modulation de faible rang :

$$\mathbf{z}_t = \text{SiLU}(\mathbf{U} \mathbf{h}_t) \odot (\mathbf{1} + \mathbf{E}[x_t]), \quad (6)$$

$$\mathbf{O}(\mathbf{h}_t, x_t) = \mathbf{D} \mathbf{z}_t. \quad (7)$$

La même table $\mathbf{E}$ est partagée entre les couches ; les matrices de projection restent propres à chaque couche. La branche est initialisée de façon neutre en mettant $\mathbf{E}$ à zéro et en employant une petite porte de sortie apprise α :

$$\mathbf{F}_{\text{objet}}(\mathbf{h}_t, x_t) = \mathbf{F}(\mathbf{h}_t) + \alpha \mathbf{O}(\mathbf{h}_t, x_t). \quad (8)$$

2.3 Résidu de micro-expert déterministe

Chaque couche affecte le token x_t à l'un des N micro-experts selon

$$e_\ell(x_t) = (x_t + \ell) \bmod N. \quad (9)$$

Toutes les activations cachées expertes sont calculées dans une même opération tensorielle régulière, puis la sortie retenue est sélectionnée sans tri ni dispersion. Pour une largeur experte w ,

$$\mathbf{R}_\ell(\mathbf{h}_t, x_t) = \beta \mathbf{W}_d^{(e_\ell(x_t))} \left[\text{SiLU}(\mathbf{W}_g^{(e_\ell(x_t))} \mathbf{h}_t) \odot \mathbf{W}_u^{(e_\ell(x_t))} \mathbf{h}_t \right]. \quad (10)$$

La sortie feed-forward complète est $\mathbf{F} + \alpha \mathbf{O} + \beta \mathbf{R}$. Il s'agit d'un résidu lexical ajouté à un chemin dense partagé, et non d'une affirmation selon laquelle le routage remplacerait le calcul commun.

TABLE 1 – Résultats principaux à nombre de tokens apparié. Les cellules restent en attente jusqu’à ce que chaque entraînement atteigne son budget enregistré et que ses métriques soient archivées.

Modèle	Paramètres	Loss tenue à part ↓	Tokens/s ↑
Baseline GQA dense	en attente	EN ATTENTE	en attente
Convolution causale, FFN dense	en attente	en attente	en attente
Convolution causale + objet lexical	en attente	en attente	en attente
Modèle lexical à micro-experts complet	98 194 244	EN ATTENTE	EN ATTENTE

3 Protocole expérimental

3.1 Contrôles appariés

L’étude principale compare quatre modèles de langage causaux d’environ 100 millions de paramètres :

1. un Transformer dense à attention groupée (GQA) et SwiGLU dense ;
2. une convolution causale avec SwiGLU dense ;
3. une convolution causale avec objet lexical partagé, sans résidu de micro-expert ;
4. le modèle complet combinant convolution causale, objet lexical partagé et micro-experts.

Le nombre de paramètres, le tokenizer, le corpus, la partition tenue à part, l’optimiseur, le calendrier du taux d’apprentissage, la précision, la longueur de séquence, le nombre effectif de tokens par batch, la seed et le nombre total de tokens d’entraînement sont maintenus constants ou explicitement rapportés lorsqu’une égalité exacte est impossible.

3.2 Données et tokenizer

Les modèles utilisent le tokenizer `o200k_base` de 200 019 entrées et FineWeb-Edu sample-10BT. Les documents dont l’indice dans le flux source est congru à zéro modulo 20 forment un flux fixe tenu à part et sont exclus de l’entraînement. Les 95% restants constituent le flux d’entraînement. Le budget principal est de 1 000 013 824 tokens d’entraînement par condition, avec des séquences de longueur 2 048.

3.3 Mesures

Nous rapportons l’entropie croisée next-token sur les données tenues à part, le nombre de paramètres, le nombre réel de tokens, le débit d’entraînement, le pic mémoire de l’accélérateur, le débit de prefill et la latence du décodage incrémental à batch un. La qualité à nombre de tokens apparié et l’efficacité en temps réel sont traitées comme des affirmations distinctes. La correction du décodage exige que les logits produits token par token avec cache correspondent aux logits de la séquence complète dans la tolérance numérique annoncée.

Nous ajoutons des diagnostics de rappel associatif : association clé-valeur, induction et apprentissage synthétique en contexte, évalués selon la distance. Ils visent à mesurer directement une faiblesse potentielle des modèles sans attention que la loss moyenne peut masquer.

3.4 Matériel

Les entraînements principaux utilisent un seul NVIDIA H100 80GB HBM3 en BF16. Les expériences Apple M5 ne sont rapportées que comme diagnostics courts et ne servent pas de preuve principale de qualité.

4 Résultats

4.1 Éléments diagnostiques

Avant les entraînements principaux, de courts diagnostics à nombre de paramètres apparié ont uniquement servi à éliminer les configurations défectueuses et à sélectionner le candidat. Sur Apple M5, le candidat

complet sans attention, comptant 98,2 millions de paramètres, a atteint une loss tenue à part de 7,557967 après 204 800 tokens, contre 7,944988 pour un contrôle dense sauvegardé ; le dernier débit d'entraînement mesuré était de 6 145 contre 4 779 tokens/s. Ces valeurs de très court terme ne constituent pas une preuve finale, car la dynamique d'optimisation initiale peut différer entre architectures.

Une extension à état récurrent diagonal persistant a également été testée. Elle a été stabilisée numériquement au moyen d'une évaluation parallèle par segments, mais s'est montrée inférieure au candidat convolutionnel plus simple en loss comme en débit aux longueurs 256 et 1 024. Comme ces longueurs dépassent peu le champ réceptif de la convolution, cette extension reste une ablation à réévaluer à 4–8k plutôt qu'une composante du modèle canonique.

4.2 Équivalence incrémentale et état fixe

Pour le modèle sélectionné, le décodage incrémental avec cache reproduit les logits de la séquence complète avec une tolérance absolue et relative de 10^{-5} dans les tests unitaires. Les adresses des caches restent stables au cours d'appels de décodage répétés. La configuration à dix couches et de largeur 384 conserve 297 216 éléments d'état par séquence, soit 580,5 Kio en BF16/FP16, indépendamment de la longueur du contexte généré. La capture CUDA Graph et les benchmarks de décodage sur le matériel cible restent en attente et ne seront pas revendiqués avant leur exécution.

5 Travaux connexes

Les convolutions causales, les modèles séquentiels à espace d'état, les alternatives récurrentes à l'attention, le calcul conditionnel et les mécanismes de mémoire lexicale motivent chacun une partie de cette étude. Nous ne revendiquons pas comme nouvelles, prises séparément, la convolution causale depthwise, l'affectation déterministe des experts, le calcul expert partagé ni les embeddings indexés par token. La question étudiée est de savoir si leur intégration contrainte offre un compromis utile entre qualité, débit et taille d'état à l'échelle des petits modèles de langage. Cette section sera complétée à partir d'une revue vérifiée des travaux les plus proches avant la soumission ; aucune référence n'est inventée dans ce premier brouillon.

6 Limites et impact élargi

L'étude actuelle se limite à environ 100 millions de paramètres, un tokenizer principal, un corpus de textes web et un nombre fini de seeds. Un résultat favorable n'établirait ni le comportement à grande échelle ni une supériorité universelle sur l'attention. Le champ réceptif convolutionnel est fini ; l'état de décodage fixe est obtenu en renonçant en partie à un accès non borné au préfixe complet. Le débit matériel dépend de l'implémentation et du dispositif et ne doit pas être généralisé à partir d'un Apple M5 ou d'un seul H100.

Le conditionnement par l'identité du token peut encoder des biais lexicaux et fréquentiels propres au corpus. FineWeb-Edu hérite des limites des grands corpus web, notamment une provenance incertaine, des déséquilibres de représentation et des contenus potentiellement nuisibles. L'architecture proposée ne supprime pas ces risques. Son inférence à état fixe pourrait réduire la mémoire et l'énergie de service, mais les gains d'efficacité peuvent aussi diminuer le coût de déploiement de modèles de faible qualité ou nuisibles. Nous rapportons la provenance des données, la construction du jeu tenu à part et les contraintes de déploiement afin que ces compromis puissent être évalués.

7 Conclusion

Nous formulons la modélisation du langage sans attention comme une séparation entre transport causal partagé et spécialisation lexicale étroite. Le candidat obtenu emploie des opérations tensorielles régulières et un état incrémental de taille fixe. Les conclusions finales sont différées jusqu'à l'achèvement de tous les contrôles appariés sur un milliard de tokens et des évaluations tenues à part. Si le candidat ne surpasse pas ses contrôles, le résultat négatif indiquera néanmoins quelles décisions d'intégration échouent à budget apparié et les affirmations seront réduites en conséquence.

Références

A Liste de contrôle pour la reproductibilité

Le supplément anonyme final contiendra :

- le code source exact du modèle et la configuration réalisée pour chaque condition rapportée ;
- le nom du tokenizer, la taille du vocabulaire, la révision du dataset et la règle déterministe de partition train/évaluation ;
- le nombre de paramètres et les assertions sur le nombre exact de tokens ;
- les métriques CSV brutes par étape et les scripts qui régénèrent les tableaux ;
- l’environnement, l’accélérateur et les versions de CUDA et PyTorch ;
- les tests de causalité, d’absence de paramètres QKV, d’identité de l’objet partagé, d’équivalence entre séquence complète et décodage incrémental, et de stabilité des adresses du cache ;
- les commandes d’entraînement et de benchmark d’inférence, sans identifiants, noms d’hôtes, chemins locaux ni historique Git.

L’artefact d’évaluation exclura les checkpoints sauf lorsqu’ils sont nécessaires à une évaluation rapportée, ainsi que l’historique Git, les métadonnées d’auteur, les adresses cloud et les logs transitoires.